CESI
ÉCOLE D'INGÉNIEURS

Livrable 4 – Systèmes embarqués

Worldwide weather watcher

Documentation technique

04/11/2024



Groupe N°4 (salle 111) :

RINN Clément
LEMBLE Lorenzo
THOMAS Paul

Sommaire :

-Documentation technique..... 3

-Fonctionnement global du système 3

-Architecture générale du programme..... 4

-Fichier principal 5

-Fichier Header 6

-Interruptions 9

- Configuration 10

-Acquisition des données 13

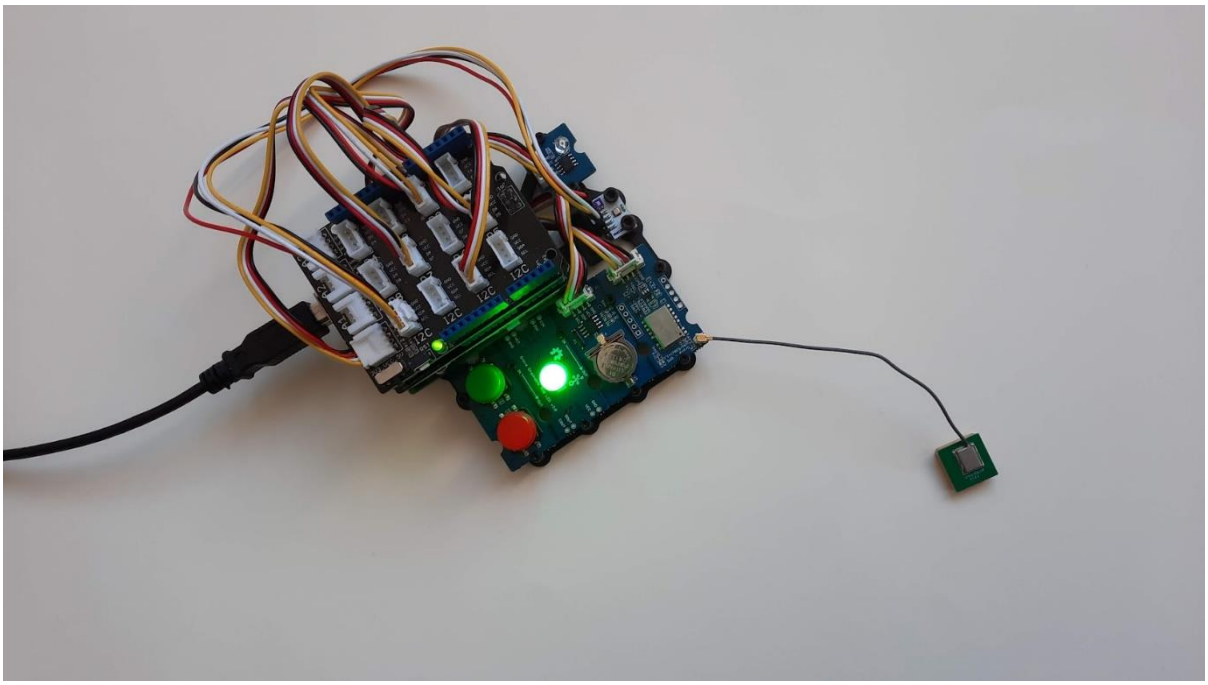
-Sauvegarde des données.....15

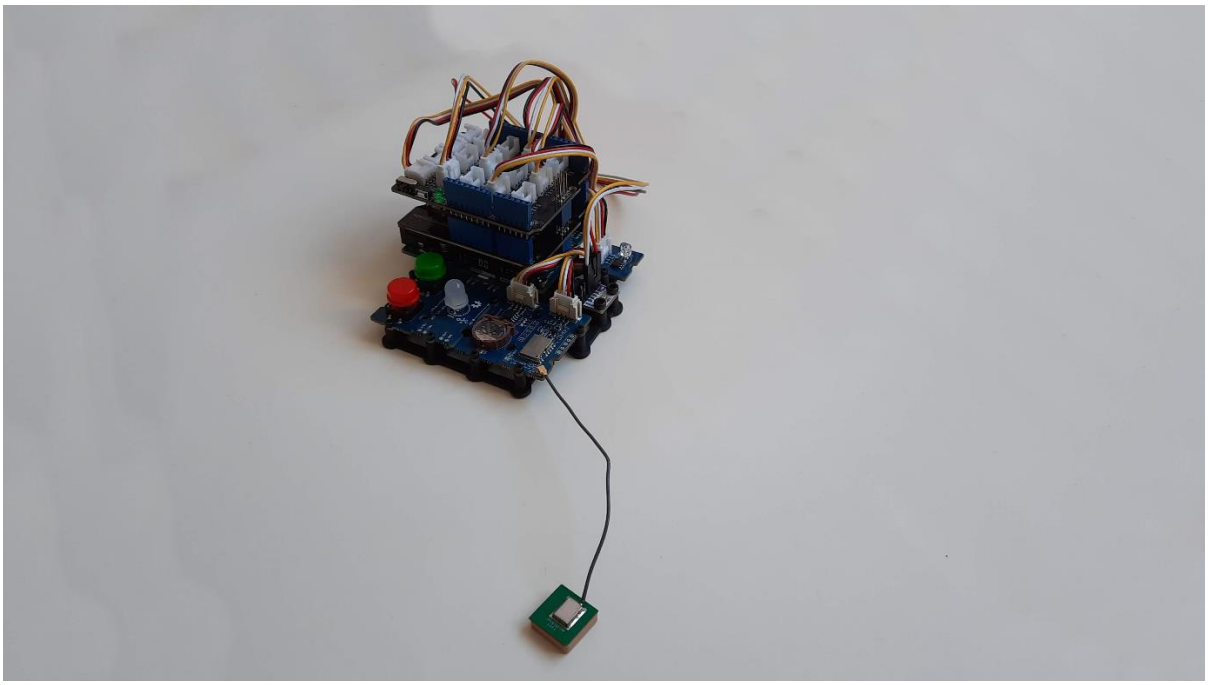
-Fonctions utiles.....16

Documentation technique

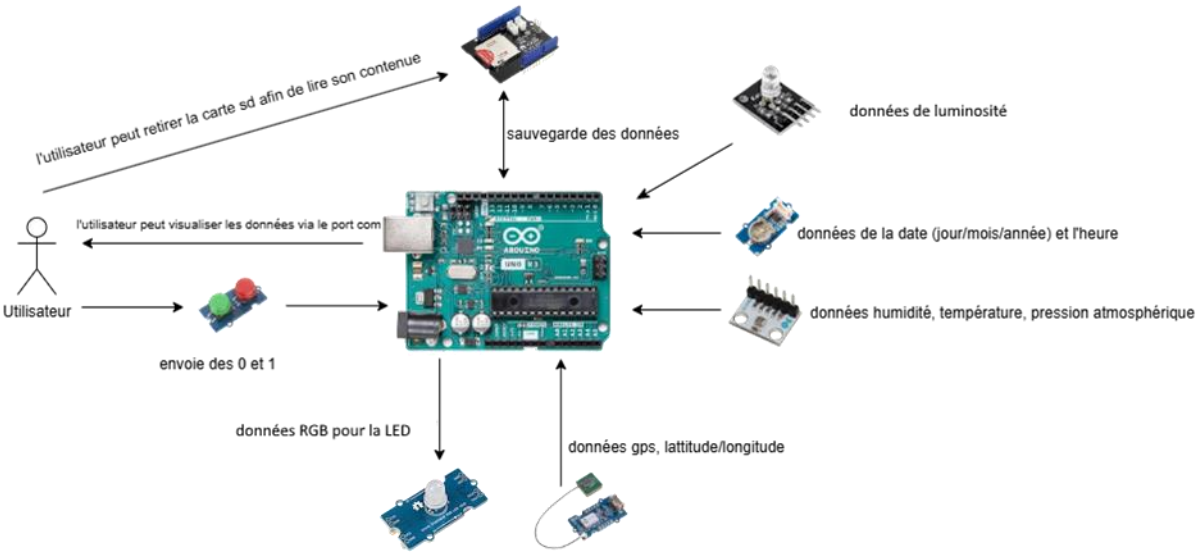
Fonctionnement global du système

Notre système WorldWide Weather Watcher est un système permettant la collecte de données météorologiques avec plusieurs modes de fonctionnements. Il comprends 1 Arduino UNO R3, deux boutons poussoirs, une LED RGB, une carte SD avec son lecteur, une horloge, un capteur GPS, un capteur de luminosité, et enfin un capteur d'humidité, de température et de pression atmosphérique.





Voici le schéma du flux de données de notre système :



Les données sont acquises par divers capteurs (température, humidité, pression, ...) et sont envoyées au microcontrôleur. Ce dernier traite les données et applique les paramètres configurés par l'utilisateur. Les résultats sont ensuite horodatés et enregistrés sur une carte SD, avec une gestion des fichiers basée sur leur taille maximale. Les données peuvent également être consultées en temps réel via une interface série. En cas d'erreur (comme une défaillance de capteur ou un problème d'accès à la carte SD), des signaux lumineux indiquent l'état du système.

Architecture générale du programme

Afin de structurer notre programme de manière claire et organisée, nous l'avons réparti en sept grandes parties distinctes. Le fichier principal (main) sert de point d'entrée, tandis que le fichier header contient les définitions nécessaires. Les interruptions gèrent les événements en temps réel, et la configuration définit les paramètres de fonctionnement. L'acquisition des données est responsable de la collecte des informations, suivie par la sauvegarde des données qui garantit leur enregistrement pour une consultation ultérieure. Enfin, les fonctions utiles regroupent des outils complémentaires nécessaires au bon fonctionnement de l'ensemble.

Fichier principal :

La partie main remplit le rôle de programme principal. C'est ce morceau du programme qui servira de colonne vertébrale à l'ensemble de notre code.

On y retrouve ainsi l'initialisation des différentes variables tel que l'état des boutons, le timer, ou encore les leds.

```
// Déclaration des variables globales, utilisées dans plusieurs fonctions (setup, loop, ...)
Button redButton = {.state: false, .pressStart: 0, .pin: 2};
Button greenButton = {.state: false, .pressStart: 0, .pin: 3};
ChainableLED led(7, 8, 1);
Config config;
DS1307 clock;
BME280 sensor;
SoftwareSerial gps(5, 6);
Mode currentMode = STANDARD;
Mode previousMode;
GpsState gpsState = WAITING;
Color blinkColor1 = GREEN;
Color blinkColor2 = OFF;
SdFat32 SD;
WeatherData data;
unsigned long gpsTimer;
unsigned long startTimer = millis();
bool longerBlink = false;
bool errors[6] = {false};
```

Vient ensuite la fonction setup, qui va définir les interruptions matérielle pour les boutons, vérifier le résultat de la fonction start afin de déterminer si le mode de départ sera le mode configuration ou le mode standard et enfin faire initialiser/démarrer certains éléments comme le gps, ou la clock et va également vérifier la présence de la carte SD.

```
void setup() {
    Serial.begin(9600);  // Initialisation de la communication série
    Serial.setTimeout(30000);  // Temps d'attente maximal pour la réception de données
    attachInterrupt(digitalPinToInterrupt(redButton.pin), redButtonInterrupt, CHANGE);  // Interruption sur le bouton rouge
    attachInterrupt(digitalPinToInterrupt(greenButton.pin), greenButtonInterrupt, CHANGE);  // Interruption sur le bouton vert

    if(starting()) {  // Si le bouton rouge est pressé pendant le démarrage
        setMode(CONFIG);  // On passe en mode configuration
    } else {
        setMode(STANDARD);  // On passe en mode standard
    }

    EEPROM.get(0, &config);  // Récupération de la configuration enregistrée dans l'EEPROM
    gps.begin(9600);  // Initialisation du GPS
    sensor.init();  // Initialisation des capteurs de température, d'humidité et de pression
    clock.begin();  // Initialisation de l'horloge
    SD.begin(4);  // Initialisation de la carte SD, sur le pin 4
}
```

Enfin la fonction loop va permettre de faire le passage entre les différents modes en fonction des actions de l'utilisateur, mais également d'appeler les différentes fonctions ou réaliser le programme lié à chaque mode. A savoir appeler la fonction checkUserInput en mode configuration (qui sera vu en détail dans la partie configuration), la multiplication par 2 de l'intervalle en mode économique, la récolte des données et l'affichage/ sauvegarde des données en fonction du mode.

```
void loop() {
    checkErrors();  // Vérification et traitement des erreurs

    if (currentMode == CONFIG) {  // Si on est en mode configuration
        checkUserInput();  // Vérification des entrées utilisateur

    } else {  // Sinon
        handleButtonPresses();  // Traitement des pressions de boutons
        const unsigned long interval = config.logInterval.value * 60000 * (currentMode == ECONOMIQUE ? 2 : 1);  // Calcul de l'intervalle de temps entre chaque enregistrement
        if (millis() - startTimer >= interval) {  // Si l'intervalle est écoulé
            processData();  // Traitement des données (acquisition + enregistrement/affichage)
        }
    }
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
setup()	Fonction appelée au démarrage de l'Arduino, initialise les composants et les variables globales.	Aucune	Aucune
loop()	Fonction appelée en boucle après setup(), gère les opérations principales de l'application.	Aucune	Aucune

Fichier header :

Le fichier header est un fichier qui permet premièrement de lier tous les fichiers du projets ensemble en déclarant toutes les fonctions appelées entre les fichiers, mais permet également de regrouper l'ensemble des données partagées par les différents fichiers, comme les structures, les énumérations et les variables qui seront utilisés dans d'autres parties du programme, tel que la structure Config regroupant de nombreux paramètres qui pourront être modifié par l'utilisateur dans la partie configuration, ou encore l'énumération Mode qui va regrouper l'ensemble des modes possible. C'est également dans ce fichier que l'on va déclarer toutes les bibliothèques et librairies utiles pour notre projet :

```
#include <Arduino.h>  // Librairie Arduino pour les fonctions de base
#include <EEPROM.h>  // Librairie pour l'écriture et la lecture dans l'EEPROM
#include <DS1307.h>  // Librairie pour l'horloge RTC
#include <ChainableLED.h>  // Librairie pour la LED RGB
#include <Seeed_BME280.h>  // Librairie pour le capteur BME280 (température, humidité, pression)
#include <SoftwareSerial.h>  // Librairie pour la communication série logicielle (GPS)
#include <SdFat.h>  // Librairie pour la carte SD (optimisée et plus légère que la librairie SD.h)
```

```
// Enumérations pour les différents états, modes, couleurs et erreurs
enum GpsState { WAITING, LOADING, FINISHED };
enum Mode { STANDARD, ECONOMIQUE, MAINTENANCE, CONFIG };
enum Color { GREEN, RED, BLUE, YELLOW, ORANGE, WHITE, OFF };
enum Error { RTC, GPS, SENSOR, DATA, SD_FULL, SD_RW };
```

```
// Déclaration des variables globales, utilisées dans plusieurs fonctions (setup, loop, ...)
extern SoftwareSerial gps;
extern ChainableLED led;
extern DS1307 clock;
extern BME280 sensor;
extern SdFat32 SD;
extern Color blinkColor1;  // Couleur 1 pour le clignotement
extern Color blinkColor2;  // Couleur 2 pour le clignotement
extern Config config;  // Configuration, avec tous les paramètres
extern Button redButton;  // Bouton rouge
extern Button greenButton;  // Bouton vert
extern Mode currentMode;  // Mode actuel
extern Mode previousMode;  // Mode précédent (pour la maintenance)
extern WeatherData data;  // Données météo
extern GpsState gpsState;  // Etat du GPS
extern bool longerBlink;  // Indique si la couleur 1 doit rester plus longtemps
extern bool errors[6];  // Tableau des erreurs
extern unsigned long gpsTimer;  // Timer pour le GPS
extern unsigned long startTimer;  // Timer pour les intervalles de log
```

```
// Structure pour les paramètres de configuration
struct Parameter {
    int value;
    int minValue;
    int maxValue;
};

// Structure pour la configuration
struct Config {
    Parameter logInterval = {.value:10, .minValue:1, .maxValue:60};  // Interval de log en minutes
    Parameter fileMaxSize = {.value:2048, .minValue:1024, .maxValue:16384};  // Taille maximale d'un fichier en octets
    Parameter timeout = {.value:30, .minValue:10, .maxValue:120};  // Timeout en secondes pour le GPS
    Parameter lumin = {.value:1, .minValue:0, .maxValue:1};  // Log de la luminosité
    Parameter luminLow = {.value:255, .minValue:0, .maxValue:1023};  // Seuil bas de luminosité
    Parameter luminHigh = {.value:768, .minValue:0, .maxValue:1023};
    Parameter tempAir = {.value:1, .minValue:0, .maxValue:1};
    Parameter minTempAir = {.value:-10, .minValue:-40, .maxValue:85};
    Parameter maxTempAir = {.value:60, .minValue:-40, .maxValue:85};
    Parameter hygr = {.value:1, .minValue:0, .maxValue:1};
    Parameter hygrMinT = {.value:0, .minValue:-40, .maxValue:85};
    Parameter hygrMaxT = {.value:50, .minValue:-40, .maxValue:85};
    Parameter pressure = {.value:1, .minValue:0, .maxValue:1};
    Parameter pressureMin = {.value:850, .minValue:300, .maxValue:1100};
    Parameter pressureMax = {.value:1080, .minValue:300, .maxValue:1100};
};

// Structure pour un bouton
struct Button {
    bool state;
    unsigned long pressStart;
    byte pin;
};

// Structure pour les données météo
struct WeatherData {
    byte hour, minute, second;
    float temperature, humidity, pressure;
    unsigned int lumin;
    float latitude, longitude;
};
```

Interruptions :

Ici sont regroupés l'ensemble des interruptions utilisés dans le programme. On y retrouve donc les fonctions initBlinkInterrupt() et stopBlinkUnterrupt() qui permettent respectivement de démarrer le clignotement des leds et de stopper le clignotement (notamment pour le démarrage et la gestion des erreurs) grâce à notre interruption ISR.

Enfin nous avons les fonctions redBouttonInterrupt et greenButtonInterrupt qui vont permettre d'actualiser létat du bouton rouge ou vert.

```
// Fonction permettant d'initialiser l'interruption pour le clignotement
void initBlinkInterrupt() {
    if(!alreadyInit) {  // Si l'interruption n'est pas déjà initialisée
        noInterrupts();  // Désactivation des autres interruptions pour éviter les problèmes
        longerState = false;  // Initialisation de l'état de clignotement
        TCCR1A = 0;  // Réinitialisation des registres
        TCCR1B = 0;  // Réinitialisation des registres
        TCNT1  = 0;  // Réinitialisation du compteur
        OCR1A = (62500 * 0.5) - 1;  // Valeur de comparaison pour une interruption toutes les 0.5 secondes
        TCCR1B |= (1 << WGM12) | (1 << CS12);  // Configuration du timer en mode CTC avec un prédiviseur de 256
        TIMSK1 |= (1 << OCIE1A);  // Activation de l'interruption
        alreadyInit = true;  // Indication que l'interruption est initialisée
        interrupts();  // Réactivation des autres interruptions
    }
}

// Fonction permettant d'arrêter l'interruption pour le clignotement
void stopBlinkInterrupt() {
    TIMSK1 &= ~(1 << OCIE1A);  // Désactivation de l'interruption
    setMode(currentMode);  // Mise à jour de la couleur de la LED
    alreadyInit = false;  // Indication que l'interruption n'est plus initialisée
}

// Interruption pour le clignotement
ISR(TIMER1_COMPA_vect) {
    setLedColor(useColor1 ? blinkColor1 : blinkColor2);  // Mise à jour de la couleur de la LED
    if(useColor1 && longerBlink && !longerState) {  // Si la couleur 1 doit rester plus longtemps
        longerState = true;  // Mise à jour de l'état
    } else {  // Sinon
        useColor1 = !useColor1;  // Changement de couleur
        longerState = false;  // Réinitialisation de l'état
    }
}
```

```
// Fonction appelée lors des interruptions sur les boutons
void buttonInterrupt(Button& button) {
    bool isPressed = digitalRead(button.pin) == LOW;  // Lecture de l'état du bouton

    if (isPressed && !button.state) {  // Si le bouton vient d'être appuyé
        button.pressStart = millis();  // Mise à jour de l'heure de début d'appui
        button.state = true;  // Mise à jour de l'état du bouton
    } else if (!isPressed) {  // Si le bouton est relâché
        button.state = false;  // Mise à jour de l'état du bouton
    }
}

void redButtonInterrupt() {  // Interruption sur le bouton rouge
    buttonInterrupt([&]redButton);
}

void greenButtonInterrupt() {  // Interruption sur le bouton vert
    buttonInterrupt([&]greenButton);
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
initBlinkInterrupt()	Initialise l'interruption pour le clignotement de la LED.	Aucune	Aucune
stopBlinkInterrupt()	Arrête l'interruption pour le clignotement de la LED.	Aucune	Aucune
ISR(TIMER1_COMPA_vect)	Fonction d'interruption appelée lorsque le timer atteint la valeur de comparaison.	Aucune	Aucune
buttonInterrupt(Button& button)	Gère l'état d'un bouton lors des interruptions.	Button& button	Aucune
redButtonInterrupt()	Interruption pour le bouton rouge.	Aucune (interdite pour les interruptions)	Aucune
greenButtonInterrupt()	Interruption pour le bouton vert.	Aucune (interdite pour les interruptions)	Aucune

Configuration :

La partie configuration a pour rôle de permettre à l'utilisateur d'entrer diverses commandes afin de changer les paramètres de l'appareil. Pour cela le programme va récupérer les commandes de l'utilisateur et en fonction de la commande utilisé va modifier les valeurs concerné à l'aide de applyCommand(), voir afficher un message d'erreur dans le cas ou aucune comande n'est reconnu. Dans le cas des commandes sans valeurs ou liés aux dates et à l'heure, la fonction checkUserInput va elle-même s'en charger, comme pour la commande Reset qui va utiliser les valeurs stockés dans Config ou la commande CLOCK qui va vérifier l'heure, les minutes et les secondes entrés par l'utilisateur sous la forme hh:mm:ss.

Enfin si le programme ne détecte aucune action de l'utilisateur au bout de 30 minutes (timeout de il repasse le mode en mode standard.

```
const char daysOfWeek[7][4] PROGMEM = {"SUN", "MON", "TUE", "WED", "THU", "FRI", "SAT"};  // Jours de la semaine

// Fonction pour mettre à jour un paramètre
void setParameter(Parameter &param, int newValue) {
    if (newValue >= param.minValue && newValue <= param.maxValue) {  // Si la nouvelle valeur est dans la plage autorisée
        param.value = newValue;  // Mise à jour du paramètre
        Serial.println(F("Paramètre mis à jour"));
    } else {
        Serial.println(F("Erreur: plage"));
    }
}

// Fonction pour appliquer une commande
void applyCommand(const char* command, int value) {
    static const struct {
        const char* name;  // Nom de la commande
        Parameter* param;  // Paramètre associé
        } commandMap[] = {  // Tableau associatif des commandes
        {.name=PSTR("LOG_INTERVAL"), .param=&config.logInterval},
        {.name=PSTR("FILE_MAX_SIZE"), .param=&config.fileMaxSize},
        {.name=PSTR("TIMEOUT"), .param=&config.timeout},
        {.name=PSTR("LUMIN"), .param=&config.lumin},
        {.name=PSTR("LUMIN_LOW"), .param=&config.luminLow},
        {.name=PSTR("LUMIN_HIGH"), .param=&config.luminHigh},
        {.name=PSTR("TEMP_AIR"), .param=&config.tempAir},
        {.name=PSTR("MIN_TEMP_AIR"), .param=&config.minTempAir},
        {.name=PSTR("MAX_TEMP_AIR"), .param=&config.maxTempAir},
        {.name=PSTR("HYGR"), .param=&config.hygr},
        {.name=PSTR("HYGR_MIN_T"), .param=&config.hygrMinT},
        {.name=PSTR("HYGR_MAX_T"), .param=&config.hygrMaxT},
        {.name=PSTR("PRESSURE"), .param=&config.pressure},
        {.name=PSTR("PRESSURE_MIN"), .param=&config.pressureMin},
        {.name=PSTR("PRESSURE_MAX"), .param=&config.pressureMax},
    };

    for (auto& cmd : commandMap) {  // Parcours du tableau des commandes
        if (!strcmp_P(command, cmd.name)) {  // Si la commande est trouvée
            setParameter([&]*cmd.param, value);  // Mise à jour du paramètre
            return;
        }
    }
    Serial.println(F("Erreur: COMMANDE"));  // Si la commande n'est pas trouvée
}
```

```
// Fonction appelée en permanence pendant le mode configuration pour vérifier les entrées utilisateur
void checkUserInput() {
    char input[32];  // Buffer pour stocker l'entrée utilisateur
    if (Serial.readBytesUntil('\n', input, sizeof(input)) > 0) {  // Si une entrée est disponible
        input[strcspn(input, "\r")] = 0;  // Suppression du caractère du retour à la ligne (pour éviter des problèmes)
        char* command = strtok(input, "=");  // Extraction de la commande
        char* value = strtok(NULL, "=");  // Extraction de la valeur

        if (!strcmp_P(command, PSTR("RESET"))) {  // Si la commande est RESET
            config = Config();  // Réinitialisation de la configuration
            Serial.println(F("Config réinitialisée"));

        } else if (!strcmp_P(command, PSTR("VERSION"))) {  // Si la commande est VERSION
            Serial.println(F("Version : v-final"));
            Serial.println(F("Lot : Groupe4"));

        } else if (!strcmp_P(command, PSTR("CLOCK")) && value) {  // Si la commande est CLOCK
            byte hours = atoi(value);  // Extraction des heures avec atoi (conversion ASCII -> entier)
            byte minutes = atoi(value + 3);  // Extraction des minutes (décalage de 3 caractères)
            byte seconds = atoi(value + 6);  // Extraction des secondes (décalage de 6 caractères)
            if (hours <= 23 && minutes <= 59 && seconds <= 59) {  // Si les valeurs sont valides
                clock.getTime();  // Récupération de l'heure actuelle
                clock.fillByHMS(hours, minutes, seconds);  // Mise à jour de l'heure
                clock.fillByYMD(clock.year + 2000, clock.month, clock.dayOfMonth);  // Récupération de la date actuelle
                clock.fillDayOfWeek(clock.dayOfWeek);  // Récupération du jour de la semaine actuel
                clock.setTime();  // Mise à jour de l'horloge
                Serial.println(F("Heure du système mise à jour."));
            } else {
                Serial.println(F("Erreur : DONNEES"));
            }

        } else if (!strcmp_P(command, PSTR("DATE")) && value) {  // Si la commande est DATE
            byte day = atoi(value);  // Extraction du jour avec atoi (conversion ASCII -> entier)
            byte month = atoi(value + 3);  // Extraction du mois (décalage de 3 caractères)
            int year = atoi(value + 6);  // Extraction de l'année (décalage de 6 caractères)

            if (day >= 1 && day <= 31 && month >= 1 && month <= 12 && year >= 2000 && year <= 2099) {  // Si les valeurs sont valides
                clock.getTime();  // Récupération de l'heure actuelle
                clock.fillByHMS(clock.hour, clock.minute, clock.second);  // Récupération de l'heure actuelle
                clock.fillByYMD(year, month, day);  // Mise à jour de la date
                clock.fillDayOfWeek(clock.dayOfWeek);  // Récupération du jour de la semaine actuel
                clock.setTime();  // Mise à jour de l'horloge
                Serial.println(F("Date du système mise à jour."));
            } else {
                Serial.println(F("Erreur : DONNEES"));
            }

        } else if (!strcmp_P(command, PSTR("DAY")) && value) {  // Si la commande est DAY
            int dayIndex = -1;  // Index du jour de la semaine
            for (int i = 0; i < 7; i++) {  // Parcours des jours de la semaine
                if (!strcmp(value, daysOfWeek[i])) {  // Si le jour est trouvé
                    dayIndex = i;  // Mise à jour de l'index
                    clock.getTime();  // Récupération de l'heure actuelle
                    clock.fillDayOfWeek(i + 1);  // Mise à jour du jour de la semaine
                    clock.fillByHMS(clock.hour, clock.minute, clock.second);  // Récupération de l'heure actuelle
                    clock.fillByYMD(clock.year + 2000, clock.month, clock.dayOfMonth);  // Récupération de la date actuelle
                    clock.setTime();  // Mise à jour de l'horloge
                    Serial.println(F("Jour de la semaine mis à jour."));
                    break;
                }
            }

            if(dayIndex == -1) {  // Si le jour n'est pas trouvé
                Serial.println(F("Erreur : DONNEES"));
            }
        } else {  // Si la commande est une commande de configuration
            applyCommand(command, value ? atoi(value) : 0);  // Application de la commande avec la fonction applyCommand
        }
        EEPROM.put(0, config);  // Enregistrement de la configuration dans l'EEPROM
    } else {  // Si la communication série est inactive (pas d'entrée depuis 30 minutes)
        Serial.println(F("Inactivité détectée"));
        setMode(STANDARD);  // Passage en mode standard
    }
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
setParameter()	Met à jour un paramètre de la configuration si la nouvelle valeur est dans la plage autorisée.	Parameter &param, int newValue	Aucune
applyCommand()	Applique une commande pour mettre à jour un paramètre associé.	Const char* command, int value	Aucune
checkUserInput()	Vérifie les entrées utilisateur en mode configuration et applique les commandes.	Aucune	Aucune

Acquisition des données :

Cette section du programme gère la collecte et le stockage des données. La fonction acquireData récolte les informations des capteurs, telles que la température, l'humidité, la pression et les coordonnées GPS (en mode standard), tout en récupérant l'heure, les minutes et les secondes au moment de la lecture. Elle vérifie également la validité des valeurs des capteurs, en enregistrant des erreurs si nécessaire. Ensuite, la fonction sauvegarderDonnees stocke les données acquises sur une carte SD. Si la taille maximale du fichier est atteinte, un nouveau fichier est créé avec la date comme nom. En cas d'absence du fichier, une erreur est déclenchée.

```
// Fonction pour acquérir les données météo
WeatherData acquireData() {  // Fonction pour acquérir les données météo
    clock.getTime();  // Récupération de l'heure actuelle
    data.hour = clock.hour;  // Récupération de l'heure
    data.minute = clock.minute;  // Récupération des minutes
    data.second = clock.second;  // Récupération des secondes

    errors[RTC] = (!data.hour || !data.minute || !data.second);  // Si l'heure n'est pas valide, erreur RTC

    if(config.lumin.value) {  // Si la luminosité est activée
        data.lumin = analogRead(A0);  // Lecture de la luminosité
        errors[SENSOR] = (data.lumin <= 0);  // Si la luminosité n'est pas valide, erreur de capteur
    }

    if(config.tempAir.value) {  // Si la température est activée
        data.temperature = sensor.getTemperature();  // Lecture de la température
        errors[DATA] = (data.temperature < config.minTempAir.value || data.temperature > config.maxTempAir.value);  // Si la température n'est pas valide, erreur de données
    }

    if(config.hygr.value) {  // Si l'humidité est activée
        data.humidity = sensor.getHumidity();  // Lecture de l'humidité
        errors[SENSOR] = (data.humidity <= 0);  // Si l'humidité n'est pas valide, erreur de capteur
    }

    if(config.pressure.value) {  // Si la pression est activée
        data.pressure = sensor.getPressure() / 100;  // Conversion de la pression en hPa
        errors[DATA] = (data.pressure < config.pressureMin.value || data.pressure > config.pressureMax.value);  // Si la pression n'est pas valide, erreur de données
    }

    gpsState = LOADING;  // Chargement des données GPS
    return data;  // Retour des données
}
```

```
// Fonction pour acquérir les données GPS
void acquireGPS() {
    if (millis() - gpsTimer > config.timeout.value * 1000) {  // Si le timeout est atteint
        gpsState = FINISHED;  // Fin de l'acquisition
        data.latitude = NULL;  // Latitude nulle
        data.longitude = NULL;  // Longitude nulle
        errors[GPS] = true;  // Erreur GPS
        return;
    }

    if (gps.available()) {  // Si des données sont disponibles
        char gpsData[100];  // Buffer pour les données GPS
        gps.readBytesUntil('\n', gpsData, sizeof(gpsData) - 1);  // Lecture des données
        gpsData[sizeof(gpsData) - 1] = '\0';  // Ajout du caractère de fin de chaîne

        if (strncmp(gpsData, "$GNGGA", 6) == 0) {  // Si la trame est une trame GNGGA
            int fieldIndex = 0;  // Index du champ
            char* field = strtok(gpsData, ",");  // Extraction du premier champ

            while (field != NULL) {
                if (fieldIndex == 2 && strlen(field) > 0) {  // Si le champ est la latitude
                    data.latitude = atof(field) / 100.0;  // Conversion de la latitude
                } else if (fieldIndex == 4 && strlen(field) > 0) {  // Si le champ est la longitude
                    data.longitude = atof(field) / 100.0;  // Conversion de la longitude
                }

                fieldIndex++;  // Incrémentation de l'index
                field = strtok(NULL, ",");  // Extraction du champ suivant
            }

            if (data.latitude != NULL && data.longitude != NULL) {  // Si les coordonnées sont valides
                gpsState = FINISHED;  // Fin de l'acquisition
                errors[GPS] = false;  // Pas d'erreur GPS
            }
        }
    }
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
acquireGPS()	Acquiert les données GPS et met à jour les coordonnées.	Aucune	Aucune
acquireData()	Acquiert les données météo et les met à jour dans la structure data.	Aucune	WeatherData

Sauvegarde des données :

Cette section du programme est responsable de la sauvegarde des données sur la carte SD. La fonction saveData commence par récupérer l'heure actuelle et génère un nom de fichier basé sur la date. Si le fichier de log existe déjà et que sa taille dépasse la taille maximale autorisée, un nouveau fichier est créé avec un index incrémental jusqu'à ce qu'un nom de fichier unique soit trouvé. Les données de l'ancien fichier sont alors copiées dans le nouveau, avant que l'ancien fichier ne soit supprimé. Ensuite, la fonction écrit les données acquises, y compris l'heure, la température, l'humidité, la pression, la luminosité et les coordonnées GPS, dans le fichier. Si les coordonnées GPS ne sont pas valides, elle enregistre "NA" à la place. La fonction ferme finalement le fichier et réinitialise l'erreur d'écriture, ou signale une erreur si le fichier n'a pas pu être ouvert.

```
// Fonction pour sauvegarder les données sur la carte SD
void saveData() {
    clock.getTime();  // Récupération de l'heure actuelle
    char fileName[] = "20xxxx_0.LOG";  // Nom du fichier de log
    fileName[2] = '0' + clock.month / 10;  // Mise à jour du mois avec les dizaines
    fileName[3] = '0' + clock.month % 10;  // Mise à jour du mois avec les unités
    fileName[4] = '0' + clock.dayOfMonth / 10;   // Mise à jour du jour avec les dizaines
    fileName[5] = '0' + clock.dayOfMonth % 10;  // Mise à jour du jour avec les unités

    File32 file = SD.open(fileName, FILE_WRITE);  // Ouverture du fichier
    if (file) {  // Si le fichier est ouvert
        if(file.size() + sizeof(data) > config.fileMaxSize.value) {  // Si la taille du fichier dépasse la taille maximale
            int fileIndex = 1;  // Index du fichier
            char newFileName[] = "20xxxx_0.LOG";  // Nouveau nom de fichier
            do {  // Tant que le fichier existe
                newFileName[7] = '0' + fileIndex++;  // Mise à jour de l'index
            } while (SD.exists(newFileName));

            File newFile = SD.open(newFileName, FILE_WRITE);  // Ouverture du nouveau fichier
            if (newFile) {  // Si le fichier est ouvert
                char buffer[32];  // Buffer pour la copie des données
                while (file.available()) {  // Tant que le fichier contient des données
                    int bytesRead = file.readBytes(buffer, sizeof(buffer));  // Lecture des données
                    newFile.write(buffer, bytesRead);  // Écriture des données dans le nouveau fichier
                }
                newFile.close();  // Fermeture du nouveau fichier
            }
            file.close();  // Fermeture de l'ancien fichier
            file.remove();  // Suppression de l'ancien fichier
            file = SD.open(fileName, FILE_WRITE);  // Réouverture de l'ancien fichier
        }

        file.print(data.hour);
        file.print(':'); file.print(data.minute);
        file.print(':'); file.print(data.second);
        file.print(';'); file.print(data.temperature);
        file.print(';'); file.print(data.humidity);
        file.print(';'); file.print(data.pressure);
        file.print(';'); file.print(data.lumin); file.print(';');

        if (data.latitude != NULL && data.longitude != NULL) {  // Si les coordonnées GPS sont valides
            file.print(data.latitude); file.print(';'); file.println(data.longitude);  // Écriture des coordonnées GPS
        } else {
            file.println(F("NA;NA;"));  // Sinon, écriture de NA pour les coordonnées GPS
        }
        file.close();  // Fermeture du fichier
        errors[SD_RW] = false;  // Réinitialisation de l'erreur d'écriture
    }
    else {
        errors[SD_RW] = true;  // Si le fichier n'est pas ouvert, erreur d'écriture
    }
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
saveData()	Sauvegarde les données dans un fichier de log sur la carte SD.	Aucune	Aucune

Fonctions utiles :

Cette dernière partie du programme gère l'ensemble des interactions et des données de la station météo. La fonction setLedColor contrôle la couleur de la LED en fonction du mode actuel, tandis que setMode met à jour ce mode et la couleur associée. La fonction checkErrors surveille les erreurs et ajuste le clignotement de la LED en conséquence. Pour le démarrage, la fonction starting vérifie si le bouton rouge est pressé dans un délai de 5 secondes, et handleButtonPresses gère les actions des boutons rouge et vert pour passer entre les modes standard, économique et maintenance. L'acquisition des données météo est traitée par processData, qui récupère les informations du GPS et sauvegarde ou affiche les données selon le mode actif. Enfin, displayData affiche toutes les données collectées, y compris l'heure, la température, l'humidité, la pression, la luminosité, et les coordonnées GPS, en signalant "NA" si les coordonnées ne sont pas disponibles.

```
// Fonction pour mettre la couleur de la LED
void setLedColor(Color color) {
    switch (color) {
        case GREEN: led.setColorRGB(0, 0, 255, 0); break;
        case RED: led.setColorRGB(0, 255, 0, 0); break;
        case BLUE: led.setColorRGB(0, 0, 0, 255); break;
        case YELLOW: led.setColorRGB(0, 255, 255, 0); break;
        case ORANGE: led.setColorRGB(0, 255, 100, 0); break;
        case WHITE: led.setColorRGB(0, 255, 255, 255); break;
        case OFF: led.setColorRGB(0, 0, 0, 0); break;
    }
}
```

```
// Fonction pour mettre le mode
void setMode(Mode mode) {
    const Color modeColors[4] = {GREEN, BLUE, ORANGE, YELLOW};
    setLedColor(modeColors[mode]);  // Mise à jour de la couleur de la LED en fonction du mode
    currentMode = mode;  // Mise à jour du mode actuel
}
```

```
// Fonction pour vérifier et traiter les erreurs
void checkErrors() {
    const Color errorColors[6] = {BLUE, YELLOW, GREEN, GREEN, WHITE, WHITE};  // Couleurs pour les erreurs
    for (int i = 0; i < 6; ++i) {  // Parcours des erreurs
        if (errors[i]) {  // Si l'erreur est détectée
            blinkColor1 = RED;  // Couleur 1 pour le clignotement
            blinkColor2 = errorColors[i];  // Couleur 2 pour le clignotement
            longerBlink = i == 3 || i == 5;  // Indique si la couleur 2 doit rester plus longtemps
            initBlinkInterrupt();  // Initialisation de l'interruption pour le clignotement
            return;
        }
    }

    stopBlinkInterrupt();  // Arrêt de l'interruption pour le clignotement si aucune erreur n'est détectée
}
```

```
// Fonction pour vérifier les entrées utilisateur
void handleButtonPresses() {
    if (redButton.state && millis() - redButton.pressStart > 5000) {  // Si le bouton rouge est pressé pendant plus de 5 secondes
        if (currentMode == MAINTENANCE) {  // Si on est en mode maintenance
            SD.begin(4);  // Redémarrage de la carte SD
            setMode(previousMode);  // Retour au mode précédent
        } else {
            SD.end();  // Arrêt de la carte SD
            previousMode = currentMode;  // Sauvegarde du mode actuel
            setMode(MAINTENANCE);  // Passage en mode maintenance
        }
        redButton.state = false;  // Réinitialisation de l'état du bouton
    } else if (greenButton.state && millis() - greenButton.pressStart > 5000) {  // Si le bouton vert est pressé pendant plus de 5 secondes
        if (currentMode != MAINTENANCE) {  // Si on n'est pas en mode maintenance
            setMode(currentMode == ECONOMIQUE ? STANDARD : ECONOMIQUE);  // Passage en mode économique ou standard
        }
        greenButton.state = false;  // Réinitialisation de l'état du bouton
    }
}
```

```
// Fonction pour démarrer le système (faire clignoter la LED et vérifier si le bouton rouge est pressé)
bool starting() {
    initBlinkInterrupt();  // Initialisation de l'interruption pour le clignotement
    unsigned long startTimer = millis();  // Timer pour le démarrage
    while (millis() - startTimer < 5000) {  // Tant que le timer n'est pas écoulé (5 secondes)
        if(redButton.state) {  // Si le bouton rouge est pressé
            stopBlinkInterrupt();  // Arrêt de l'interruption pour le clignotement
            return true;  // Retourne vrai (le bouton rouge est pressé)
        }
    }
    stopBlinkInterrupt();  // Arrêt de l'interruption pour le clignotement
    return false;  // Retourne faux (le bouton rouge n'est pas pressé)
}
```

```
// Fonction pour acquérir les données météo
void processData() {
    if (gpsState == WAITING) {  // Si le GPS est en attente
        acquireData();  // Acquisition des données météo
        gpsTimer = millis();  // Mise à jour du timer GPS
    } else if (gpsState == LOADING) {  // Si le GPS est en chargement
        acquireGPS();  // Vérifier si le GPS a reçu des données
    } else {  // Si le GPS a fini de charger
        gpsState = WAITING;  // Retour à l'état d'attente
        startTimer = millis();  // Mise à jour du timer
        if (currentMode == MAINTENANCE) {  // Si on est en mode maintenance
            displayData();  // Affichage des données
        } else {
            saveData();  // Sauvegarde des données
        }
    }
}
```

```
// Fonction pour afficher toutes les données météo dans le moniteur série
void displayData() {
    Serial.print(F("Heure : ")); Serial.print(data.hour);
    Serial.print(':'); Serial.print(data.minute);
    Serial.print(':'); Serial.print(data.second);
    Serial.print(F(" | Température : ")); Serial.print(data.temperature);
    Serial.print(F("°C | Humidité : ")); Serial.print(data.humidity);
    Serial.print(F("% | Pression : ")); Serial.print(data.pressure);
    Serial.print(F("hPa | Luminosité : ")); Serial.print(data.lumin);
    if (data.latitude == NULL || data.longitude == NULL) {  // Si les coordonnées GPS ne sont pas valides
        Serial.println(F(" | Latitude : NA | Longitude : NA"));  // Affichage de NA
    } else {
        Serial.print(F(" | Latitude : ")); Serial.print(data.latitude);
        Serial.print(F(" | Longitude : ")); Serial.println(data.longitude);
    }
}
```

Récapitulatif des fonctions :

Fonction	Description	Variables en paramètres	Variables de retour
setLedColor()	Définit la couleur de la LED en fonction de la valeur passée.	Color color	Aucune
setMode()	Met à jour le mode actuel et change la couleur de la LED en fonction du mode.	Mode mode	Aucune

checkErrors()	Vérifie les erreurs et fait clignoter la LED en fonction des erreurs détectées.	Aucune	Aucune
starting()	Démarre le système et vérifie si le bouton rouge est pressé pendant 5 secondes.	Aucune	bool
handleButtonPresses()	Gère les pressions sur les boutons rouge et vert pour changer de mode ou redémarrer la carte SD.	Aucune	Aucune
processData()	Acquiert et traite les données météo en fonction de l'état du GPS.	Aucune	Aucune
displayData()	Affiche toutes les données météo dans le moniteur série.	Aucune	Aucune